

EXPERIMENT 3: IMAGE TRANSFORMS IN FREQUENCY DOMAIN

AIM:

To perform image transforms in frequency domain using python in google colab.

MATERIALS REQUIRED:

Google colab software (python)

CODE:

Theory:

CV2.imread()-path to image file as an argument and returns image as numpy array.

CV2.cvtColor()-easier to work with grayscale images.

CV2.dft()-compute discrete fourier transform of image.

Numpy.fft shift()-shift the zero frequency component of fourier transform.

Numpy abs()-compute magnitude of discrete fourier transform.

1)FFT

```
import cv2
```

```
import numpy as np
```

```
# now we will be loading the image and converting it to grayscale
```

```
image = cv2.imread("chestxray.jpg")
```

```
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
# Compute the discrete Fourier Transform of the image
```

```
fourier = cv2.dft(np.float32(gray), flags=cv2.DFT_COMPLEX_OUTPUT)
```

```
# Shift the zero-frequency component to the center of the spectrum
```

```
Fourier_shift = np.fft.fftshift(fourier)
```

```
# calculate the magnitude of the Fourier Transform
```

```
magnitude = 20*np.log(cv2.magnitude(fourier_shift[:, :, 0], fourier_shift[:, :, 1]))
```

```
# Scale the magnitude for display
```

```
magnitude = cv2.normalize(magnitude, None, 0, 255, cv2.NORM_MINMAX, cv2.CV_8UC1)
```

8UC1)

Display the magnitude of the Fourier Transform

cv2.imshow('Fourier Transform', magnitude)

cv2.waitKey(0)

cv2.destroyAllWindows()

2)IFFT

import numpy as np

import cv2

from matplotlib import pyplot as plt

Load grayscale image

image = cv2.imread("chestxray.jpg")

if image is None:

raise ValueError("Image not found")

----- FORWARD FFT -----

fft = np.fft.fft2(image)

fft_shift = np.fft.fftshift(fft)

----- OPTIONAL FILTER (low-pass) -----

rows, cols = image.shape

crow, ccol = rows // 2, cols // 2

mask = np.zeros((rows, cols), np.uint8)

mask[crow-30:crow+30, ccol-30:ccol+30] = 1

fft_filtered = fft_shift * mask

----- INVERSE FOURIER TRANSFORM -----

ifft_shift = np.fft.ifftshift(fft_filtered)

image_ifft = np.fft.ifft2(ifft_shift)

Take magnitude (real image)

image_ifft = np.abs(image_ifft)

```
# ----- DISPLAY -----
plt.figure(figsize=(12,4))
plt.subplot(131)
plt.imshow(image, cmap='gray')
plt.title("Original Image")
plt.axis("off")
plt.subplot(132)
plt.imshow(np.log(1 + np.abs(fft_shift)), cmap='gray')
plt.title("FFT Spectrum")
plt.axis("off")
plt.subplot(133)
plt.imshow(image_ifft, cmap='gray')
plt.title("Reconstructed Image (IFFT)")
plt.axis("off")
plt.show()
```

OUTPUT:



3)DFT

```
import numpy as np
import cv2
from matplotlib import pyplot as plt
# read the input image
# you can specify the path to image
```

```

image_path = "chestxray.jpg"
image = cv2.imread(image_path, 0)
# calculating the discrete Fourier transform
DFT = cv2.dft(np.float32(image), flags=cv2.DFT_COMPLEX_OUTPUT)
# reposition the zero-frequency component to the spectrum's middle
shift = np.fft.fftshift(DFT)
row, col = image.shape
center_row, center_col = row // 2, col // 2
# create a mask with a centered square of 1s
mask = np.zeros((row, col, 2), np.uint8)
mask[center_row - 30:center_row + 30, center_col - 30:center_col+30]
# put the mask and inverse DFT in place.
fft_shift = shift * mask
fft_ifft_shift = np.fft.ifftshift(fft_shift)
imageThen = cv2.idft(fft_ifft_shift)
# calculate the magnitude of the inverse DFT
imageThen = cv2.magnitude(imageThen[:, :, 0], imageThen[:, :, 1])
# visualize the original image and the magnitude spectrum
plt.figure(figsize=(10,10))
plt.subplot(121), plt.imshow(image, cmap='gray')
plt.title('Input Image'), plt.xticks([]), plt.yticks([])
plt.subplot(122), plt.imshow(imageThen, cmap='gray')
plt.title('Magnitude Spectrum'), plt.xticks([]), plt.yticks([])
plt.show()

```

4) IDFT

```

import cv2
import numpy as np

```

```

import matplotlib.pyplot as plt
import os

# Define the correct path to your image
image_path = "chestxray.jpg"
img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

if img is None:
    print(f"Load failed: Could not read the image from '{image_path}'. Check if the path is
    correct and the file exists.")
else:
    h, w = img.shape

    # Pad the image to optimal DFT size
    img_pad = cv2.copyMakeBorder(img, 0, cv2.getOptimalDFTSize(h)-h, 0,
    cv2.getOptimalDFTSize(w)-w, cv2.BORDER_CONSTANT)
    img_f = np.float32(img_pad)/255.0

    # Perform DFT
    dft = cv2.dft(img_f, flags=cv2.DFT_COMPLEX_OUTPUT)
    dft_shift = np.fft.fftshift(dft)

    # Calculate and display magnitude spectrum
    magnitude = 20*np.log(cv2.magnitude(dft_shift[:, :, 0], dft_shift[:, :, 1]))

    plt.figure(figsize=(15, 6))
    plt.subplot(131)
    plt.imshow(img, cmap='gray')
    plt.title('Original Image')
    plt.axis('off')

    plt.subplot(132)
    plt.imshow(magnitude, cmap='gray')
    plt.title('DFT Spectrum')
    plt.axis('off')

```

```

# Perform IDFT
dft_ishift = np.fft.ifftshift(dft_shift)
img_ift = cv2.idft(dft_ishift, flags=cv2.DFT_SCALE | cv2.DFT_REAL_OUTPUT)
# Crop back to original size and convert to uint8
img_ift = np.clip(img_ift*255, 0, 255).astype('uint8')[:h, :w]

plt.subplot(133)
plt.imshow(img_ift, cmap='gray')
plt.title('Reconstructed Image (IDFT)')
plt.axis('off')
plt.tight_layout()
plt.show()

```

OUTPUT:



RESULT:

Thus the image transforms in frequency domain were performed by google colab.